

DSSAD 完整架构、调用链路与测试指南

一、设计架构

1.1 代码分层

DSSAD 采用**两层架构**：平台公共层 + 车型适配层。

平台公共代码（gb_dssad/）		车型适配代码（dssad_cfg/）	
提供框架、不感知具体信号		所有信号/事件/数据表在此定义	
edr_data_process.cpp	主循环入口	dssad_data_get.c/h	数据获取函数实现
dssad_event_trigger.c/h	事件检测引擎	dssad_time_segment_cfg.c	5张JSON表定义
dssad_data_pack.c/h	JSON序列化+发送	dssad_time_stamp_cfg.c	二进制表+事件发送
dssad_common.c/h	函数注册+工具宏	dssad_func_register.c	函数指针绑定
dssad_if.h	对外接口	dssad_base_cfg.h	数据最大长度
cJSON.c/h	JSON库	CMakeLists.txt	编译配置
		DssadBaseData_20.h	协议头+flat struct

1.2 依赖注入模式

平台层通过全局结构体 gDssadConfig（DssadConfig_t 类型）持有 14 个函数指针 + 1 个配置值。车型代码在 dssad_func_register.c 中填写这些函数指针，调用 DssadFunConfig_Resiger() 注册。平台代码只通过 gDssadConfig.xxx() 调用，不直接依赖任何车型信号。

```
[dssad_func_register.c]
sDssadConfig = {
    .dssad_get_collision_status = xxx,
    .dssad_time_stamp_data_send = xxx,
    ...
}
dssad_func_register() _____

[dssad_common.c]
DssadFunConfig_Resiger(&sDssadConfig)
→ 逐个拷贝函数指针到 gDssadConfig
→ 校验所有指针非NULL
→ 设置 product_register_flag

[edr_data_process.cpp]
if (gDssadConfig.product_register_flag == true) ← 校验通过才运行
```

1.3 通信架构

DSSAD 运行在 **GE 进程**（dji_ad_app 或 dji_bf_app），属于 **DCOS_DOMAIN**，通信底层为 DJI DCOS 发布/订阅中间件。

```
信号输入：
CAN 信号 → SigIf_Get_*( ) 同步读取（x_dom_can_rt_sigif.h）
DCMS 回调 → 7 个 topic 订阅，异步接收（感知/GNSS/DMM/SOMEIP/DTC/工厂参数）

信号输出：
时间戳事件 → dcms_mcu_topic_send(DCMS_TOPIC_FL_OPT_SEND) → FL 立即落盘
时间段数据 → dcms_mcu_topic_send_msg(6个topic) → FL 选择性留存
```

二、CMake 编译链接链路

```
src/dsar_app/CMakeLists.txt          ← 顶层入口
|
├─ include(consys_bf.cmake)           ← product_build_marcos() 宏定义
├─ include(dsar_plat_bf.cmake)        ← X_DOM_CAN_RT_SRC 平台SDK路径
|
└─ add_subdirectory(product/faw/fawhq_p301/)
    └─ CMakeLists.txt → product_build_marcos()
        |
        └─ include(fawhq_p301_config.cmake) ← CONFIG_APP_DSSAD=ON
            |
            └─ if(CONFIG_APP_DSSAD):
                |
                └─ include(dssad_cfg/dssad_cfg_config.cmake)
                    |
                    └─ include(dssad_cfg/CMakeLists.txt)
                        |
                        └─ DSSAD_COMIF_PATH = ../proxy/x_dom_can_rt_gen_com
                            |
                            └─ 源文件: gb_dssad/ (5个) + dssad_cfg/ (4个)
                                |
                                └─ add_library(app_ge_dssad_${product_name} STATIC ...)
                                    |
                                    └─ 静态库, 未链接
                    |
                    └─ add_subdirectory(proxy/)
                        |
                        └─ x_dom_can_rt_gen_com → x_dom_can_rt_gen_lib_com ← CAN 信号实现
                            |
                            └─ x_dom_someip_rt_gen_com → x_dom_someip_rt_gen_lib_com ← SOMEIP 实现
                    |
                    └─ add_subdirectory(app_core/)
                        |
                        └─ app_core/CMakeLists.txt
                            |
                            └─ add_subdirectory(app_ge_core)
                                |
                                └─ app_ge_core/CMakeLists.txt
                                    |
                                    └─ add_library(GE_CORE_TARGET_NAME STATIC ...)
                                        |
                                        └─ target_link_libraries(GE_CORE
                                            |
                                            | PRIVATE mini_dcos plat_bf_ge) ← 基底
                                            |
                                            └─ target_link_libraries(GE_CORE
                                                |
                                                | PRIVATE app_ge_dssad_${product}) ← DSSAD 链入
                                                |
                                                └─ → APP_CORE_LIB += GE_CORE_TARGET_NAME
                                    |
                                    └─ consys_bf.cmake:149: | 最终符号解析发生在 COM SO:
                                        |
                                        └─ product_static_lib_list =
                                            |
                                            ... dcos_dcms
                                            |
                                            └─ ${APP_CORE_LIB} ← (含 app_ge_dssad)
                                                |
                                                └─ x_dom_can_rt_gen_lib_com (SigIf_Get_* 实现)
                                                    |
                                                    └─ x_dom_someip_rt_gen_lib_com (SOMEIP 实现)
                                    |
                                    └─ → add_library(${PRODUCT_NAME}_com SHARED ...)
                                        |
                                        └─ → target_link_libraries(... --whole-archive ${product_static_lib_list} ...)
                                            |
                                            └─ → ${PRODUCT_NAME}_com.so

entry/CMakeLists.txt:
add_executable(dji_ad_app ...)
→ 运行时加载 com.so, DSSAD 运行在 dji_ad_app 进程 (DCOS_DOMAIN)
```

三、完整调用链路

3.1 初始化

```
dssad_init()
└─ add_period_work_with_node_name(
    "dssad", "dssad", task_mid,
    APP_PERIOD_50MS,
    thread_dssad_process, nullptr)
```

[edr_data_process.cpp:108]

← 注册为 50ms 周期任务

3.2 主循环 (50ms)

```
thread_dssad_process()                                [edr_data_process.cpp:69]
|
| 前200周期: 跳过 (10秒延迟启动, 避开开机瞬态)
|
| sleep_time_compensate(run_cycle_ms)                 ← 休眠时间补偿 [-5, 5]ms
|
| if (product_register_flag == true):                 ← 函数注册校验
|
| ① dssad_common_process(run_ms)                      ← 框架初始化 (EXEC_ONCE)
|
| ② dssad_trigger_process(run_ms)                     ← ★ 事件检测 (每50ms)
|   |
|   | EXEC_ONCE: dssad_trigger_init()                 ← 注册调试回调
|   |   dcms_mcu_topic_setup_callback(
|   |     DCMS_DSSAD_TRIGGER_DEBUG,
|   |     dssad_trigger_debug_callback, ...)
|   |
|   | dssad_event_status_update()                     ← 读取 10 类事件状态
|   |   |
|   |   | [调试模式] 从 dssad_event_debug_data[] 取值
|   |   | [正常模式] 通过 gDssadConfig 函数指针取值:
|   |   |   | collision / lock / collision_risk      (时间段触发状态)
|   |   |   | ADS_Activate[32]                      (时间戳事件)
|   |   |   | ADS_Exit[32]                          (时间戳事件)
|   |   |   | InterventionRequest                  (时间戳事件)
|   |   |   | MinimumRiskStrategy                  (时间戳事件)
|   |   |   | ADS_SeriousFailure[64]                (时间戳事件)
|   |   |   | ADS_ActivatedByDriver                 (时间戳事件)
|   |   |   | ADS_ExitByDriver                      (时间戳事件)
|   |   |
|   |   | dssad_time_segment_trigger_status_report() ← 上报3个事件状态给FL
|   |   |   | dcms_mcu_topic_send_msg(
|   |   |     DCMS_TOPIC_DSSAD_EVENT_CHECK,
|   |   |     &time_segment_trigger_status)         ← 3字节二进制
|   |   |
|   |   | dssad_time_stamp_event_trigger_check()     ← 7个事件边沿检测
|   |   |   | (每个 check 函数: 比对该事件 last/cur 状态)
|   |   |   |
|   |   |   | time_stamp_event_ADS_Auto_Activate_check()
|   |   |   | time_stamp_event_ADS_Auto_Exit_check()
|   |   |   | time_stamp_event_InterventionRequest_check()
|   |   |   | time_stamp_event_MinimumRiskStrategy_check()
|   |   |   | time_stamp_event_ADS_SeriousFailure_check()
|   |   |   | time_stamp_event_ADS_ActivateByDriver_check()
|   |   |   | time_stamp_event_ADS_ExitByDriver_check()
|   |   |   |
|   |   |   | [检测到 false→true 边沿时]:
|   |   |   |   gDssadConfig.dssad_time_stamp_data_send(event_id, ...)
|   |   |   |   | 设置 Event_Id, Ads_Sys_Fail ← 仅2个赋值
|   |   |   |   | memcpy 预填充的 dssad_time_stamp_data → DssadBaseData
|   |   |   |   | dcms_mcu_topic_send(DCMS_TOPIC_FL_OPT_SEND, ...) → FL
|   |   |
|   | ③ dssad_data_get_process(run_ms)                ← 车型自定义(可为空)
|   |
|   | ④ dssad_data_pack_process(run_ms)               ← ★ 数据打包 (两个独立节奏)
|   |   |
|   |   | EXEC_ONCE: dssad_data_pack_init()
|   |   |
|   |   | [每50ms检查] dssad_time_segment_data_update()
```

```

|   |   for 每张表:
|   |   |   if (time_ms % 表的report_cycle != 0) continue; ← 没到期跳过
|   |   |   for 每个字段: 现场调 getter() 取最新值
|   |   |   dssad_time_segment_data_pack() → cJSON序列化
|   |   |   dcms_mcu_topic_send_msg() → FL
|   |   |
|   |   └─ 实际发送节奏: 表2/3/4/5 每100ms, 表1 每1000ms
|
└─ [每500ms] dssad_time_stamp_data_update()
    |   for 18个字段: 现场调 getter() 填充 dssad_time_stamp_data
    |
    └─ ★ 只刷新缓冲区, 不发送任何东西
        (发送由上面的边沿检测触发, 读取此缓冲区)

```

3.3 两种数据刷新机制对比

这是理解 DSSAD 设计的关键差异:

时间段 (time_segment) – "即采即发"

时间戳 (time_stamp) – "预热快照"

50ms 入口检查

```

|
├─ 100ms 到期?
|   └─ 调 Dssad_Get_VehSpeed() ← 7次 getter
|       └─ cJSON 序列化
|           └─ dcms_mcu_topic_send_msg() → FL
|
├─ 1000ms 到期?
|   └─ 调 Dssad_Get_VIN() ← 18次 getter
|       └─ cJSON 序列化
|           └─ dcms_mcu_topic_send_msg() → FL
|

```

数据 = 发送时刻的实时值

500ms 后台刷新

```

|
├─ 调 Dssad_Get_VIN() ← 18次 getter
├─ 调 Dssad_Get_Longitude()
├─ ...
└─ 填充到 dssad_time_stamp_data 缓冲区

```

50ms 边沿检测

```

|
├─ 发现 false→true
└─ dssad_time_stamp_data_send()
    └─ Event_Id = xxx ← 仅2个赋值
        └─ memcpy 缓冲区
            └─ dcms_mcu_topic_send() → FL

```

数据 = 500ms前到此刻之间的某个快照

关键区别:

- **时间段**: getter 调用和发送是**同步的**, 数据就是采集时刻的值
- **时间戳**: getter 调用 (500ms) 和发送 (事件触发) 是**解耦的**, 发送时直接 memcpy 预填好的缓冲区, 不在发送路径上调用 getter
- 时间戳允许滞后是因为记录的是 VIN/版本/时间/GPS/里程等慢变量, 真正关键的事件 ID 和故障码在发送那一刻赋值, 这两个不滞后

四、时间段记录 (time_segment) 详解

4.1 设计理念

"report the time segment event status. the trigger part is in data-mining. So we just record the status here."

— [dssad_event_trigger.c:332-334](#)

DSSAD 负责**周期性地**把**全部数据以 JSON 形式发出**, 不在本地做落盘决策。FL (Fleet Learning) 模块收到后, 根据 trigger_status (collision/lock/collision_risk) 决定保留哪些时间窗口的数据。

4.2 3个触发状态

状态	判断逻辑	实现位置
collision (碰撞)	滑动窗口内 X或Y 方向加速度积分速度变化 > 8.0 km/h	dssad_data_get.c:289-298
lock (锁车)	X方向速度增量 > 25.0 km/h 或 airbag crash signal = 1	dssad_data_get.c:301-310
collision_risk (碰撞风险)	AEB减速度 > 5.0 m/s² 或 SACC减速度 > 5.0 m/s² 或 InterventionRequest = 3	dssad_data_get.c:277-286

4.3 5张JSON数据表

表	名称	周期	字段数	Topic	典型内容
表1	veh_base_data	1000ms	18	/dssad/base_data_pack/v1	VIN, HW/SW版本, SN, 时间, 经纬度, 里程
表2	veh_dynamic_data	100ms	7	/dssad/odo_data_pack/v1	车速, 纵向/横向加速度, 横摆角速度
表3	ads_runtime_data	100ms	27	/dssad/adas_req_pack/v1	ADS请求的档位/转向/制动/灯光/雨刮
表4	driving_env_data	100ms	54	/dssad/fused_object/v1	9个感知目标(类型/距离/速度)
表5	driver_handle_data	100ms	8	/dssad/driver_handle_pack/v1	安全带/踏板/方向盘转角/脱手

4.4 发送机制：即采即发

```
入口：dssad_data_pack_process() 每50ms调用一次
├─ dssad_time_segment_data_update(time_ms)
│   └─ for i = 0..4: // 遍历5张表
│       └─ if (time_ms % table[i].report_cycle != 0) → continue (跳过)
│           └─ for j = 0..member_num: // ★ 现场调getter取最新值
│               └─ cur_table_data[j].data_get_function(&buffer, data_type)
│                   └─ dcms_mcu_get_curr_sensor_usec() // 取传感器时间戳
│                       └─ dssad_time_segment_data_pack() // cJSON序列化 → dcms_mcu_topic_send_msg()
```

- getter 调用与 JSON 序列化、发送在**同一个函数调用链**中，数据即当前时刻值
- 各表独立节奏：表1每1000ms触发一次，表2/3/4/5每100ms触发一次
- 如果某次50ms入口没有任何表到期，则整个函数几乎零开销（只做5次取模判断）

4.5 JSON格式示例

每条 JSON 消息结构：

```
{
  "sensor_usec": "1717334400500000",
  "type": "dssad::veh_dynamic_data_pack_t",
  "data": {
    "VehSpeed": 60.5,
    "Acc_X": 0.12,
    "Acc_Y": -0.05,
    ...
  }
}
```

五、时间戳记录 (time_stamp) 详解

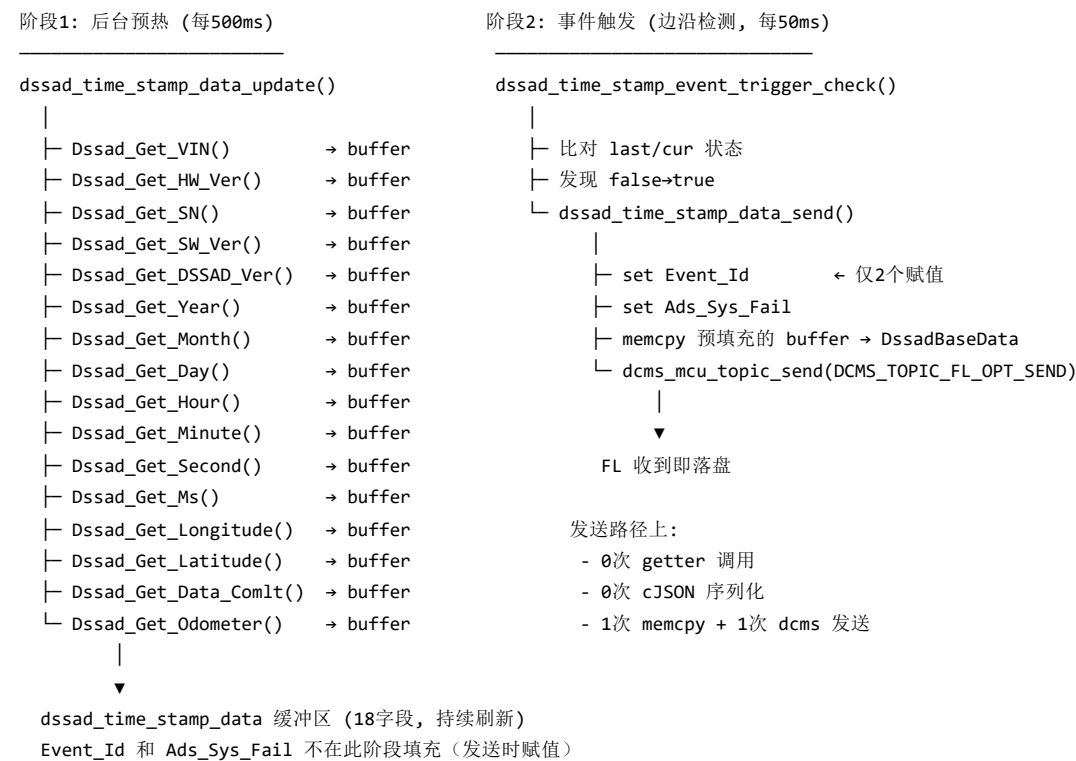
5.1 设计理念

"time stamp event check. if trigger, report the (base data + protocol header). the trigger part is check here. so in data-mining, it just need to analysis protocol header and then save."
— [dssad_event_trigger.c:355-358](#)

DSSAD 自己做边沿检测，检测到事件后立即发送带协议头的二进制快照，FL 收到即存。

时间戳采用***预热快照 + 即时发送***两阶段设计，目的是把"慢的采集"和"快的发送"解耦：

5.2 两阶段机制



为什么预热16个字段但只赋值2个？ VIN、版本、时间、GPS、里程都是慢变量（秒级变化），500ms滞后可以接受。真正关键的是 Event_Id（哪个事件触发）和 Ads_Sys_Fail（哪个DTC）——这两个必须在边沿检测到的那一瞬间赋值，不能提前填。

为什么不能在发送时现场调 getter？ 事件检测在50ms路径上（dssad_trigger_process），getter 中可能有 CAN 信号读取、浮点运算，18个 getter 叠加影响实时性。预热方案让发送路径极轻（2次赋值+1次memcpy+1次dcms发送）。

5.3 7种触发事件

EventId	事件名	触发逻辑	子事件
0x01	ADS自动激活	32个子功能任一个 last==false && cur==true	32 (NOD/TJA/ACC/ILCA/AEB/ESA/HPA/APA/NRP/TBA/MOD/MEB/Remote...)
0x02	ADS自动退出	同上（退出子事件）	32
0x03	介入请求	last < cur （0→1→2→3）	无
0x04	最小风险策略	last==false && cur==true	无

EventId	事件名	触发逻辑	子事件
0x05	ADS严重失效	64个DTC任一个边沿触发，每次只报一个	64
0x08	驾驶员操作系统激活	last==false && cur==true	无
0x08	驾驶员操作系统退出	last==false && cur==true	无

5.3 触发后发送的数据

发送格式 (DssadBaseData_t):

meta_data[1578] 协议头 (FL工具生成, 用于自解析)	18 个表1字段 (flat binary) VIN[36] HW_Ver[36] SN[36] SW_Ver[36] DSSAD_Ver[36] Event_Id Year Month Day Hour Minute Second Ms Longitude Latitude Data_Comlt Odometer Ads_Sys_Fail
--	--

发送函数: dcms_mcu_topic_send(DCMS_TOPIC_FL_OPT_SEND, ...)
Topic: /fl/service/operational_data

5.4 边沿检测示例

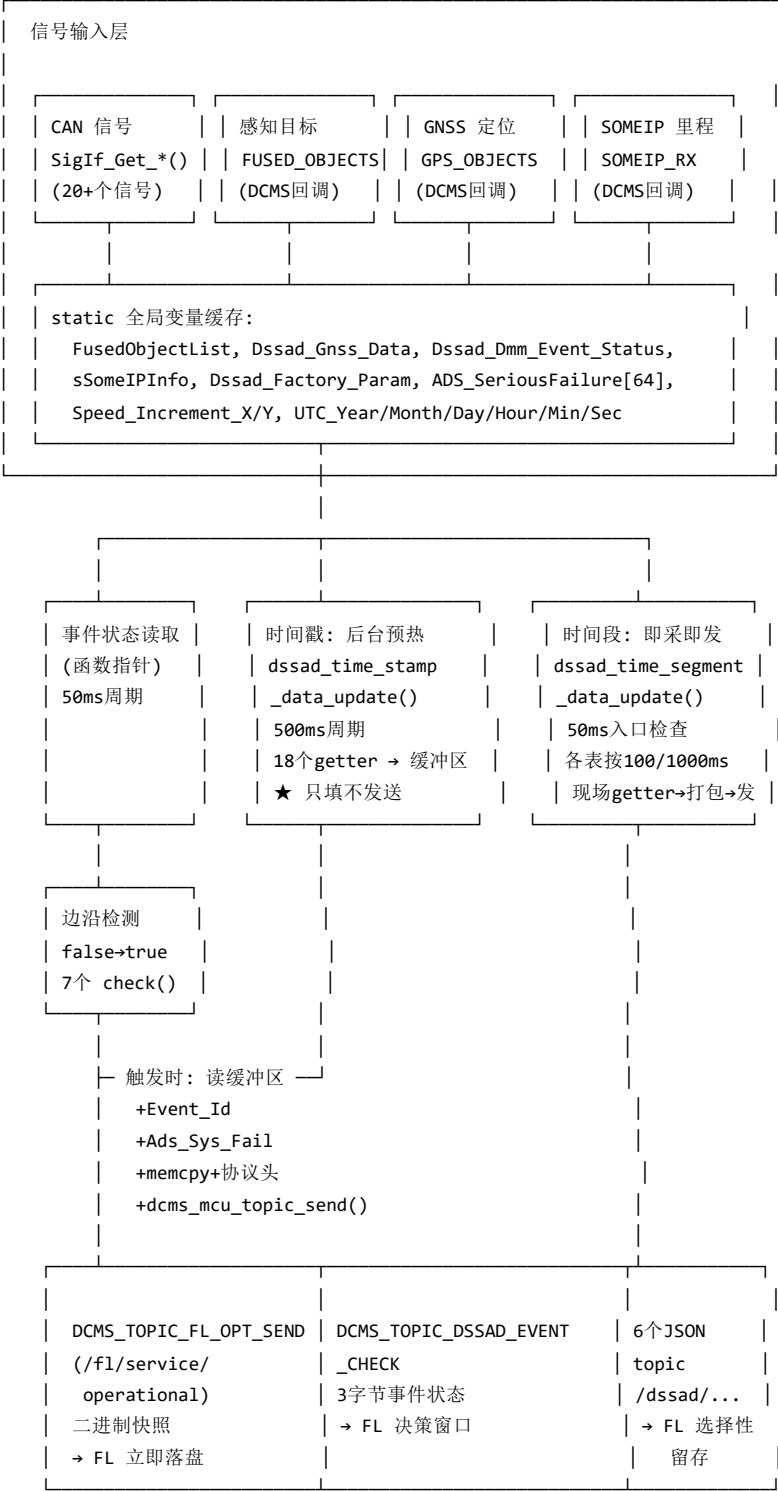
```
// ADS激活检测 (dssad_event_trigger.c:203-219)
for (i = 0; i < ADS_SUBEVENT_MAX; i++)
{
    if ((time_stamp_trigger_status_last.ADS_Activate[i] == false) &&
        (time_stamp_trigger_status.ADS_Activate[i] == true)) // false→true
    {
        gDssadConfig.dssad_time_stamp_data_send(
            EventId_ADS_Activate, "EventId_ADS_Activate",
            i, ads_activate_subevent_name[i].name); // 发送快照
    }
    time_stamp_trigger_status_last.ADS_Activate[i] =
        time_stamp_trigger_status.ADS_Activate[i]; // 记录本次状态
}
```

六、两种记录模式对比

	时间戳事件 (time_stamp)	时间段数据 (time_segment)
记录哪些数据	仅表1 (18个基础字段)	全部5张表 (114+字段)
数据格式	二进制 flat struct + 1578字节协议头	JSON (cJSON序列化)
触发决策方	DSSAD 自己做边沿检测	FL 模块根据 trigger_status 决策
触发方式	事件驱动, 边沿触发 false→true	状态持续上报, 周期性
发送时机	瞬间触发, 立即发送	按表周期: 100ms/500ms/1000ms
发送接口	dcms_mcu_topic_send()	dcms_mcu_topic_send_msg()
Topic 数量	1个	6个 (5表 + 1事件状态)

	时间戳事件 (time_stamp)	时间段数据 (time_segment)
FL 侧行为	收到即存 ("save at once")	根据 trigger_status 选择性保留窗口
数据刷新方式	500ms后台预热，发送时直接 memcpy	每次发送前现场调 getter，即采即发
发送路径开销	2次赋值 + 1次memcpy + 1次send	N次getter + cJSON序列化 + send_msg
数据时效性	最旧500ms (Event_Id/Ads_Sys_Fail实时)	当前时刻实时值
表1重叠	是 — 两模式都记录表1	

七、数据流全景图



八、数据信号清单

8.1 表1 — 车辆基础数据 (18字段)

字段	类型	数据来源
VIN	char[36]	工厂参数 (DCMS回调, 跨域来自UDS/MINIDCOS)
HW_Ver	char[36]	工厂参数
SN	char[36]	工厂参数
SW_Ver	char[36]	工厂参数
DSSAD_Ver	char[36]	硬编码 "1.00.01"
Event_Id	int32	事件触发时设置 (0x01-0x08)
Year/Month/Day	int32	gettimeofday() → UTC
Hour/Minute/Second	int32	gettimeofday() → UTC
Ms	int32	(未实现)
Longitude	float	GNSS (弧度→度)
Latitude	float	GNSS (弧度→度)
Data_ComlIt	int32	硬编码 1
Odometer	float	SOMEIP (DCMS回调)
Ads_Sys_Fail	int32	事件触发时设置 (DTC索引)

8.2 表2 — 车辆动态数据 (7字段)

字段	数据来源
VehSpeed	CAN: SigIf_Get_VehicleSpeed_oABS_1_0xc0_odefault
VehSpeed_VaiId	CAN: 同上
Acc_Y	CAN: SigIf_Get_Acceleration_Y_oPDCM_YRS_0x134_odefault
Acc_X	CAN: SigIf_Get_Acceleration_X_oPDCM_YRS_0x134_odefault
YawRate_ACU	CAN: SigIf_Get_YawRate_ACU_oYAW_1_0x124_odefault
RollRate_ACU	CAN: 同上结构体
YawAngle	CAN: 同上结构体

8.3 表3 — ADS运行数据 (27字段)

字段	数据来源
TargetGearReq	DMM运行时数据 (DCMS回调)
LateralAccReq	DMM运行时数据
VDC_SteeringWheelAngleReq	CAN: VDC系列信号
ADV_StrWhlAngReq	CAN: ADV系列信号
StrCurvtReq, FrontWhlAngReq	DMM运行时数据

字段	数据来源
EPS1_StrWhlTorqueReq, EPS1_StrWhlSpeedReq	DMM运行时数据
VehSpeedReq, LongAccReq	DMM运行时数据
AccPedalPosReq, BrakePresReq	DMM运行时数据
HAD_DrvTorqueReq, HAD_HCUSpeedControlReq	DMM运行时数据
HAD_AutoLightModeReq, BO_ExtLitAutoLitSt	DMM运行时数据
BO_HighBeamSt, HAD_HazardWarnLampReq	DMM运行时数据
HAD_LeftTurnLampReq, HAD_RightTurnLampReq	DMM运行时数据
HAD_WiperAutoLeverReq	DMM运行时数据

8.4 表4 — 环境感知数据 (9目标×6字段=54)

每个目标 (Obj1-Obj9): Type, Dist_X_Front, Dist_Y_Front, Spd_X, Spd_Y, Dist_X_Rear
数据来源: DCMS回调 DCMS_TOPIC_FUSED_OBJECTS (/sys/fused_object_list/v5)

8.5 表5 — 驾驶员操作数据 (8字段)

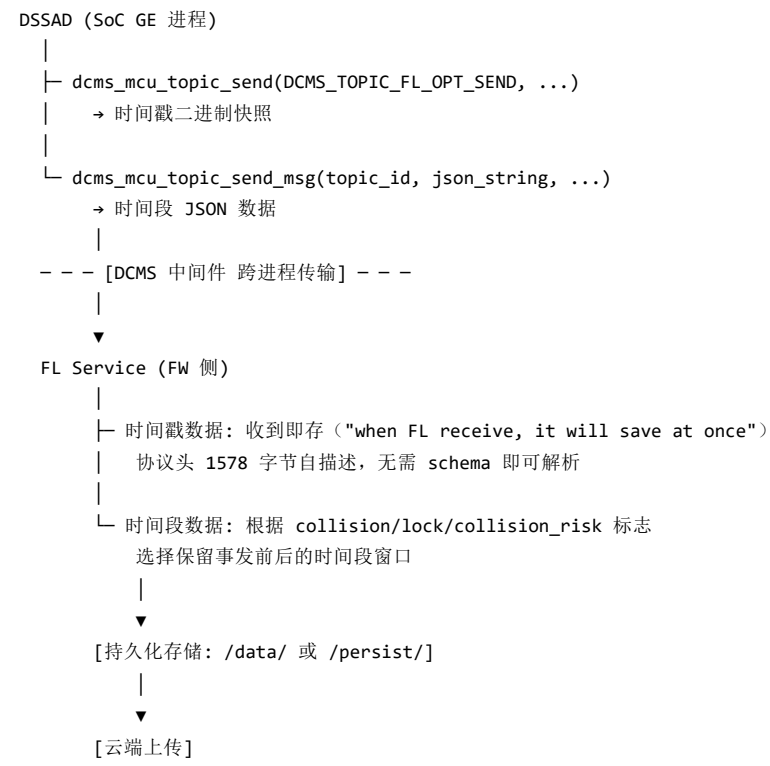
字段	数据来源
EPS_1_HandOffStatus	CAN: SigIf_Get_EPS_1_HandOffStatus_oEPS_1_1_0x150_odefault
SeatBeltSt_DSCU	CAN: SigIf_Get_SeatBeltSt_DSCU_oDSCU_1_0x244_odefault
Sf_SeatOccupySt	DMM运行时数据 (DCMS回调)
HCU_AccPedalPos	CAN: SigIf_Get_HCU_AccelerationPedalPosition_oHCU_2_2_0x92_odefault
BrakePedalPos	DMM运行时数据
IBC_BrakePedalStatus	CAN: SigIf_Get_IBC_BrakePedalStatus_oIBC_1_0xc1_odefault
EPS1_StrWhlAng	CAN: EPS系列信号
EPS_1_HandSteeringTorque	DMM运行时数据

九、落盘存储

9.1 DSSAD 不直接写文件

DSSAD 源码中没有任何 fopen / fwrite / fprintf 等文件 I/O 操作。所有持久化工作由 FL (Fleet Learning) 模块完成。

9.2 数据落盘路径



9.3 验证落盘的方法

方法	操作
查看 DSSAD 日志	事件触发时打印: <DSSAD>dssad_time_stamp_event trigger, event_id:0x01...
订阅 DCMS topic	用 topic_recoder 工具订阅输出 topic, 确认有数据
查看 FL 日志	FL 收到 DCMS_TOPIC_FL_OPT_SEND 时应打印存储日志
检查存储目录	取决于 FL 配置, 通常在持久化分区

十、测试方法

10.1 调试注入机制

DSSAD 内置了调试开关 `dssad_event_debug_flag` ([dssad_event_trigger.c:31-32](#))。开启后所有事件状态从 `dssad_event_debug_data[32]` 数组读取, 不读真实信号。

Topic: `/dssad/trigger_debug/v1` (`DCMS_DSSAD_TRIGGER_DEBUG`)
消息格式: `[byte_index(uint8), value(uint8)]` (2字节)

10.2 调试 Byte Index 映射表

Index	枚举名	对应事件	取值含义
0	Debug_None	(保留)	
1	Debug_ADS_Activate	ADS自动激活	bit位对应32个子事件

Index	枚举名	对应事件	取值含义
2	Debug_ADS_Exit	ADS自动退出	bit位对应32个子事件
3	Debug_InterventionRequest	介入请求	0→1→2→3 递增
4	Debug_MinimumRiskStrategy	最小风险策略	0/1
5	Debug_ADS_SeriousFailure	ADS严重失效	bit位对应64个DTC
6	Debug_Collision	碰撞	0/1
7	Debug_RiskOfCollision	碰撞风险	0/1
8	Debug_ADS_ActivatedByDriver	驾驶员激活	0/1
9	Debug_ADS_ExitedByDriver	驾驶员退出	0/1
10	Debug_Lock	锁车	0/1
0xFF	(特殊)	开关调试模式	0=关闭, 1=开启

10.3 测试时间戳事件

步骤1: 开启调试模式

```
send(/dssad/trigger_debug/v1, [0xFF, 0x01])
```

预期: 日志打印 byte 位映射表

步骤2: 模拟 ADS 激活 (NOD子事件)

```
send(/dssad/trigger_debug/v1, [1, 0x01])
```

预期: 每50ms检测到 false→true

日志: "<DSSAD>dssad_time_stamp_event trigger, event_id:0x01
(EventId_ADS_Activate) subevent_id 0(Byte_NOD_Activate) ret 0"

输出: DCMS_TOPIC_FL_OPT_SEND 收到二进制数据

步骤3: 模拟碰撞

```
send(/dssad/trigger_debug/v1, [6, 0x01])
```

预期: 日志 "COLLISON status: 1"

DCMS_TOPIC_DSSAD_EVENT_CHECK 收到事件状态

步骤4: 模拟介入请求 (0→1)

```
send(/dssad/trigger_debug/v1, [3, 0x01])
```

预期: 事件触发, 发送 EventId_InterventionRequest

步骤5: 模拟介入请求 (1→2, 递增触发)

```
send(/dssad/trigger_debug/v1, [3, 0x02])
```

预期: 再次触发 (last<cur 即 1<2)

步骤6: 关闭调试模式

```
send(/dssad/trigger_debug/v1, [0xFF, 0x00])
```

预期: 日志 "exit dssad debug"

10.4 测试时间段 JSON 数据

时间段数据依赖 DCMS 回调收到的真实信号数据。需要向 DSSAD 订阅的 topic 注入模拟数据:

```
// 注入感知目标 → 表4有数据
send(/sys/fused_object_list/v5, mock_fused_objects)

// 注入GNSS → 表1经纬度有值
send(/sys/fusion_gnss/v1, mock_gnss_data)

// 注入工厂参数 → 表1 VIN/SN/版本有值
send(/dssad/faw_factory_param/v1, mock_factory_param)

// 注入DTC → 表1 Ads_Sys_Fail有值
send(/dssad/ads_failure_dtc/v2, mock_dtc)

// 验证：订阅输出 topic
subscribe(/dssad/base_data_pack/v1)      → 1000ms 收到JSON
subscribe(/dssad/odo_data_pack/v1)       → 100ms  收到JSON
subscribe(/dssad/adas_req_pack/v1)       → 100ms  收到JSON
subscribe(/dssad/fused_object/v1)        → 100ms  收到JSON
subscribe(/dssad/driver_handle_pack/v1)  → 100ms  收到JSON
```

10.5 端到端验证清单

#	检查项	方法	预期
1	DSSAD初始化	查日志	dssad_common_init , dssad_trigger_init , dssad_data_pack_init
2	函数注册成功	查日志	<success> DSSAD_CFG_SUCCESS: product_func_register success
3	时间段JSON产出	订阅表2 topic	每100ms收到JSON车速/加速度数据
4	时间戳事件触发	注入并查日志	<DSSAD>dssad_time_stamp_event trigger
5	FL收到二进制数据	订阅 /fl/service/operational_data	收到带协议头的二进制包
6	碰撞事件上报	注入并订阅 event_check	收到 collision=true
7	调试模式开关	注入 [0xFF,0x01] 再 [0xFF,0x00]	日志显示进入/退出调试

十一、关键常量

常量	值	含义
DSSAD_THREAD_START_MS	10000ms	开机延迟启动（避开瞬态）
DSSAD_THREAD_CYCLE_MS	50ms	主循环周期
DSSAD_THREAD_MAX_MS	~4000000000ms	run_ms 回绕保护
ADS_SUBEVENT_MAX	32	ADS激活/退出子事件最大数
ADS_SERIOUS_DTC_MAX	64	严重失效DTC最大数
DSSAD_DATA_BUFF_SIZE	36	单个数据字段最大字节数
DSSAD_TABLE_LIST_NUM	5	时间段数据表数量
DSSAD_TIME_STAMP_CFG_NUM	18	时间戳数据字段数

十二、关键文件索引

平台公共代码 (gb_dssad/):

app_core/app_ge_core/gb_dssad/	
└─edr_data_process.cpp	主循环入口 + 时间补偿
└─dssad_event_trigger.h	事件ID + 子事件枚举 + 状态结构体
└─dssad_event_trigger.c	事件状态读取 + 7个边沿检测 + 调试回调
└─dssad_data_pack.h	数据打包接口
└─dssad_data_pack.c	JSON序列化(cJSON) + 发送 + 主调度
└─dssad_common.h	DssadConfig_t结构体 + 工具宏 + 数据联合体
└─dssad_common.c	函数指针注册校验 (DssadFunConfig_Resiger)
└─dssad_if.h	对外接口声明
└─cJSON.c/h	JSON库

车型适配代码 (dssad_cfg/):

product/faw/fawhq_p301/dssad_cfg/	
└─dssad_data_get.h	所有 Dssad_Get_* 函数声明 + 事件getter声明
└─dssad_data_get.c	数据获取实现 (CAN+DCMS回调+事件判断逻辑)
└─dssad_time_segment_cfg.c	5张JSON表定义 + 时间片数据更新
└─dssad_time_segment_cfg.h	表列表宏
└─dssad_time_stamp_cfg.c	二进制数据表 + 事件发送函数
└─dssad_time_stamp_cfg.h	时间戳配置
└─dssad_func_register.c	14个函数指针绑定
└─dssad_base_cfg.h	DSSAD_DATA_MAX_LEN
└─DssadBaseData_20.h	协议头 meta_data + flat struct
└─CMakeLists.txt	编译配置
└─dssad_cfg_config.cmake	include(CMakeLists.txt)

编译链路关键文件:

consys/consys_bf.cmake	product_build_marcos() 宏
dsar_plat_bf.cmake	Conan平台包桥接 + X_DOM_CAN_RT_SRC
app_core/CMakeLists.txt	APP_CORE_LIB 组装
app_core/app_ge_core/CMakeLists.txt	GE_CORE_TARGET_NAME 组装 + DSSAD链接
product/faw/fawhq_p301/fawhq_p301_config.cmake	CONFIG_APP_DSSAD=ON
entry/CMakeLists.txt	可执行程序构建